Project 1: Large-Scale Data Engineering for AI

Sergi Flores, Paula Esteve, Claudia Gallego

April 12, 2026

Due to the significant size of the generated data files and Spark environments, the full source code, configuration files, and implementation details are hosted on GitHub. You can access the complete repository at the following link: https://github.com/paulaa2/bda-practica1

1 Project Overview

The goal of this project is to build a complete an end-to-end Data Science pipeline. It is implemented in five distinct sections, each with its specific function to ensure a structured flow of data from collection to analysis.

- **Landing Zone:** Raw data is ingested from multiple sources, stored without transformations, ensuring the data is available for further processing and ready for the next stages.
- **Formatted Zone:** The raw data undergoes homogenization by being transformed into a standard format, ensuring all data sources adhere to a consistent structure.
- **Trusted Zone:** Data quality checks are performed, and cleaning processes are applied to eliminate errors, duplicates, and inconsistencies, ensuring the integrity of the data.
- **Exploitation Zone:** Data from the Trusted Zone is integrated, combining datasets, resolving discrepancies, and transforming data to make it ready for analysis.
- **Data Analysis Zone:** Data is processed for model training and validation, where machine learning algorithms are applied, and their performance is assessed. The goal is to develop models that accurately predict the target variable or provide insightful descriptive analysis.

2 Data

For this project, we chose to center it around the healthcare domain, specifically focusing on cardiovascular diseases. Our goal was to evaluate the risk of developing such diseases based on both medical factors, like blood pressure and cholesterol levels, and behavioral factors, such as smoking habits. To achieve this, we selected three datasets from three different sources.

The first dataset, **Cardiovascular Heart Diseases**, contains a set of health and lifestyle attributes collected during medical examinations. The target variable, cardio, indicates the presence or absence of cardiovascular disease.

The **Heart Disease Health Indicators** dataset comes from the Behavioral Risk Factor Surveillance System (BRFSS), an annual health-related telephone survey by the CDC. The survey gathers information on health behaviors, chronic conditions, and the use of preventative services. The original dataset contained responses from over 441,000 individuals across 330 features. However, we chose to work with a subset of 250,000 responses and 21 variables plus the

target.

Finally, the **Cleveland Heart Disease dataset** focuses on patient data related to heart disease diagnosis. It includes demographic and clinical features such as age, sex, chest pain type, cholesterol levels, electrocardiographic results, and more. The target variable, condition, indicates whether or not a patient has heart disease.

3 Architecture and Data Zones

3.1 Part 1: Landing Zone

The Landing Zone ingests three configured Kaggle datasets and stores each extraction as a timestamped raw Parquet snapshot. This run-based layout supports traceability and reproducibility while keeping ingestion separate from downstream transformations and serves as a stable reference point.

Ingestion is orchestrated with Airflow through a daily DAG (02:00) that iterates over the dataset list and creates one task per dataset. Each task retrieves Kaggle credentials from Airflow Variables, initializes Spark, runs the collect routine, and shuts Spark down. Operational robustness is improved through retries and execution timeouts. Dataset tasks are independent, so failures are retried automatically, although unresolved errors may still require manual follow-up.

3.2 Part 2: Formatting Zone

The Formatting Zone standardizes the data to make sure that the three source datasets are consistent and structured in the same way. Spark is used because of its efficient processing of large datasets and together with the script formatting_pipeline.py allows th application of changes so the datasets are aligned at the schema level. Then, we use DuckDB to store the formatted data for easy access and a lightweight storage solution.

As mentioned, the main focus of formatting is on standarization of syntax: making sure the names, types, and structure of the data are consistent across all datasets. Multiple formatting transformations are applied to the columns of the datasets, all described in the tables in annex.

3.3 Part 3: Trusted Zone

The Trusted Zone is the layer of the data pipeline, where strict data-quality constraints and rules are enforced on the standardized data. This zone ensures that the data meets necessary quality standards before moving to the next stages.

The inputs are the three formatted datasets that are exported to a staging Parquet area. Then, Spark applies a "clean and validate" function, checking the different types of rules. Rows that pass all rules are saved as trusted data, while those that fail are moved to quarantine. The final output includes clean Parquet snapshots of trusted data, the quarantine dataset formed by the rows that did not pass validation, and a quality report detailing rule violations. The data and metadata are then saved in a DuckDB schema for easy querying.

The rules applied cover various aspects of data quality, from ensuring non-null values to enforcing specific ranges, uniqueness, and cross-field consistency. This are:

- **Required "not null" (presence) rules**: Essential columns must contain non-null values. If a column is missing, the row is considered invalid and moved to quarantine.

- **Required range (numeric plausibility) rules**: Numeric columns must fall within a specified range. These rules ensure that values like age, blood pressure, and BMI are within realistic limits.
- **Optional range (null OR within bounds) rules**: These allow for null values, but if a value is provided, it must fall within a defined range. This ensures that data like income or mental health scores are either missing or plausible if present.
- **Required domain (categorical code / allowed set) rules**: Categorical columns must contain only predefined values. For example, gender can only be 'female' or 'male', and chest pain type must be one of a set of valid codes.
- **Cross-field consistency rules**: These rules enforce logical consistency across multiple columns. For instance, systolic blood pressure must be greater than or equal to diastolic blood pressure.
- **Uniqueness rules**: Key columns, such as 'patient_id' or 'record_key', must be unique across the dataset. Duplicate entries are quarantined to ensure data integrity.

3.4 Part 4: Exploitation Zone

In the Exploitation Zone, trusted data is integrated and processed to create the combined dataset used in analysis. Exporting the mentioned Trusted data into staging Parquet files is the first step. These files are then read by Spark, which maps each dataset to a canonical schema. Any discrepancies between datasets are resolved, and the data is transformed into the unified structure. The data is then aggregated into a singular dataset, thanks to a vertical union. Profile summaries and versioned Parquet snapshots are also agregated for traceability. Finally, the cleaned data is stored in exploitation.duckdb,.

The Canonical Schema of the final dataset, named as risk_model_input, contains the following columns, not including the profile and parquet snapshots.

Column Name	Column Name	Column Name
source_dataset	age_years	systolic_bp
source_record_id	gender	diastolic_bp
weight_kg	height_cm	bmi
cholesterol_proxy	glucose_proxy	smoking_proxy
pulse_pressure	hypertension_stage1_flag	target

3.5 Part 5: Analysis Zone

The Analysis Zone is executed by `analysis_pipeline.py` and trains two reproducible *integrated* machine learning pipelines over the canonical table `exploitation.risk_model_input`.

Both analytical pipelines follow the same execution template. The data is split into train/test partitions using stratified sampling on the target to deal with class imbalance. To deal with missings, numerical features are imputed using the median and standardized, while categorical and boolean features are imputed using the mode and one-hot encoded. Model selection is done by comparing two models, a class-balanced Logistic Regression and a class-weighted Random Forest, with stratified cross-validation. Each model is trained and validated across multiple folds that preserve the class ratio, and the model with the highest mean cross-validated score is selected.

For the selected model, the Analysis Zone chooses a custom decision threshold by maximizing F1 on training probabilities, instead of using the default 0.5. The model is then evaluated on the

held-out test set. Finally, the stage saves the trained model and generates JSON reports with performance metrics, confusion matrix, and other relevant information about its performance.

The main diference between the two pipelines is feature scope and the selection metric used for cross-validation. The `core` pipeline focuses on a compact feature set and selects the best model by ROC-AUC to emphasize general separability. While the `enriched` pipeline expands the feature space with additional health measures and selects the best model by PR-AUC (average precision) to better reflect positive-class performance under imbalance.

3.5.1 Model results

Both pipelines selected Random Forest (rf_balanced) as the model. The core pipeline achieved a best cross-validation score of 0.8201. On the other hand, enriched pipeline achieved a best score of 0.6328. The decision threshold for the first pipeline was set at 0.6389, meanwhile the enriched threshold had a value of 0.6055.

Core Test Performance	
Accuracy	0.7875
Precision	0.4347
Recall	0.6067
F1	0.5065
ROC-AUC	0.8171
PR-AUC	0.4928

Table 2: Core Model Test Performance

Enriched Test Performance	
Accuracy	0.8369
Precision	0.5396
Recall	0.6285
F1	0.5807
ROC-AUC	0.8694
PR-AUC	0.6252

Table 3: Enriched Model Test Performance

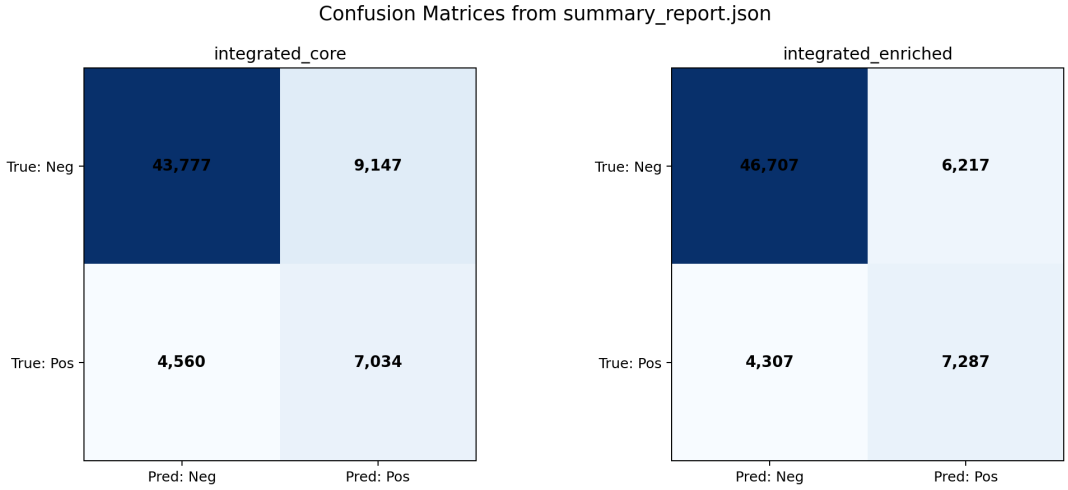

Figure 1: Confusion matrix

4 Conclusions

The implemented solution provides a full production-like path from raw ingestion to model evaluation. The zone-based design proved effective for modularity, while canonical exploitation tables simplified analytical development.The project demonstrates how data engineering and data science can be integrated through disciplined layering, as the final architecture is not only functional for the assignment, but also aligned with real-world principles.

A Annex

Dataset: `cardiovascular_disease`

Orig. Name	Orig. Type	Derived	Name	Type
id	VARCHAR		patient_id	STRING
age	VARCHAR		age_years	INT
(derived)		age_years	age_group_code	INT
gender	VARCHAR		gender	STRING
height	VARCHAR		height_cm	DOUBLE
weight	VARCHAR		weight_kg	DOUBLE
(derived)		height, weight	bmi	DOUBLE
ap_hi	VARCHAR		systolic_bp	DOUBLE
ap_lo	VARCHAR		diastolic_bp	DOUBLE
cholesterol	VARCHAR		cholesterol_level	INT
gluc	VARCHAR		glucose_level	INT
smoke	VARCHAR		is_smoker	BOOLEAN
alco	VARCHAR		drinks_alcohol	BOOLEAN
active	VARCHAR		is_active	BOOLEAN
cardio	VARCHAR		has_cardiovascular_disease	BOOLEAN

Dataset: `heart_disease_health_indicators`

Orig. Name	Orig. Type	Derived	Name	Type
(derived)		all raw columns	respondent_id	STRING
Age	DOUBLE		age_group_code	INT
(derived)		age_group_code	age_years_proxy	DOUBLE
Sex	DOUBLE		gender	STRING
BMI	DOUBLE		bmi	DOUBLE
HighBP	DOUBLE		high_blood_pressure_flag	BOOLEAN
HighChol	DOUBLE		high_cholesterol_flag	BOOLEAN
CholCheck	DOUBLE		chol_check_recent_flag	BOOLEAN
Smoker	DOUBLE		smoking_flag	BOOLEAN
Stroke	DOUBLE		stroke_history_flag	BOOLEAN
PhysActivity	DOUBLE		physical_activity_flag	BOOLEAN
Fruits	DOUBLE		fruits_daily_flag	BOOLEAN
Veggies	DOUBLE		veggies_daily_flag	BOOLEAN
HvyAlcoholConsump	DOUBLE		heavy_alcohol_flag	BOOLEAN
AnyHealthcare	DOUBLE		any_healthcare_flag	BOOLEAN
NoDocbcCost	DOUBLE		no_doctor_cost_flag	BOOLEAN
GenHlth	DOUBLE		general_health_score	INT
MentHlth	DOUBLE		mental_unhealthy_days	DOUBLE
PhysHlth	DOUBLE		physical_unhealthy_days	DOUBLE
DiffWalk	DOUBLE		difficulty_walking_flag	BOOLEAN
Education	DOUBLE		education_level	INT
Income	DOUBLE		income_level	INT
HeartDiseaseorAttack	DOUBLE		has_heart_disease	BOOLEAN

Dataset: `heart_disease_cleveland`

Orig. Name	Orig. Type	Derived	Name	Type
(derived)		all raw columns	record_key	STRING
age	INTEGER		age_years	INT
(derived)		age_years	age_group_code	INT
sex	INTEGER		gender	STRING
cp	INTEGER		chest_pain_type	INT
trestbps	INTEGER		resting_bp	DOUBLE
chol	INTEGER		serum_cholesterol	DOUBLE
fbs	INTEGER		fasting_blood_sugar_high	BOOLEAN
restecg	INTEGER		resting_ecg	INT
thalach	INTEGER		max_heart_rate	DOUBLE
exang	INTEGER		exercise_induced_angina	BOOLEAN
oldpeak	DOUBLE		st_depression	DOUBLE
slope	INTEGER		st_slope	INT
ca	INTEGER		num_major_vessels	INT
thal	INTEGER		thalassemia_type	INT
condition	INTEGER		has_heart_disease	BOOLEAN